

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

#1. Exploratory Data Analysis

#a. Load Data

```
df= pd.read_csv('/content/Titanic.csv')
df
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	
0	892	0	3	Kelly, Mr. James	male	34.5	0	0	
1	893	1	3	Wilkes, Mrs. James (Ellen Needs)	female	47.0	1	0	
2	894	0	2	Myles, Mr. Thomas Francis	male	62.0	0	0	
3	895	0	3	Wirz, Mr. Albert	male	27.0	0	0	
4	896	1	3	Hirvonen, Mrs. Alexander (Helga E Lindqvist)	female	22.0	1	1	3
...	...	...	...	...	...	...	...	...	
413	1305	0	3	Spector, Mr. Woolf	male	NaN	0	0	A.
414	1306	1	1	Oliva y Ocana, Dona. Fermina	female	39.0	0	0	PO
415	1307	0	3	Saether, Mr. Simon Sivertsen	male	38.5	0	0	SOTC 3
416	1308	0	3	Ware, Mr. Frederick	male	NaN	0	0	
417	1309	0	3	Peter, Master. Michael J	male	NaN	1	1	
418 rows × 12 columns									

#b. Data cleaning

handle missing values

```

# Check for missing values before handling
print("Missing values before handling:")
print(df.isnull().sum())

# Handle missing values
# Fill 'Age' with the median
df['Age'] = df['Age'].fillna(df['Age'].median())

# Fill 'Fare' with the median
df['Fare'] = df['Fare'].fillna(df['Fare'].median())

# Fill 'Embarked' with the mode (the most frequent value)
df['Embarked'] = df['Embarked'].fillna(df['Embarked'].mode()[0])

# Fill 'Cabin' with 'Missing' due to a large number of missing values
df['Cabin'] = df['Cabin'].fillna('Missing')

# Check for missing values after handling
print("\nMissing values after handling:")
print(df.isnull().sum())

```

Missing values before handling:

```

PassengerId    0
Survived        0
Pclass         0
Name           0
Sex            0
Age           86
SibSp          0
Parch          0
Ticket         0
Fare           1
Cabin         327
Embarked        0
dtype: int64

```

Missing values after handling:

```

PassengerId    0
Survived        0
Pclass         0
Name           0
Sex            0
Age            0
SibSp          0
Parch          0
Ticket         0
Fare           0
Cabin          0
Embarked        0
dtype: int64

```

```

# hadnle outliers

```

```

# Function to detect and handle outliers using the IQR method
def handle_outliers_iqr(df, column):

```

```

Q1 = df[column].quantile(0.25)
Q3 = df[column].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

# Cap the outliers
df[column] = df[column].clip(lower=lower_bound, upper=upper_bound)
print(f"Outliers in '{column}' handled using IQR capping.")
return df

# Identify numerical columns for outlier handling (excluding PassengerId as it
numerical_cols = ['Age', 'Fare', 'SibSp', 'Parch']

print("Original descriptive statistics for Fare:")
print(df['Fare'].describe())

# Visualize 'Fare' distribution before outlier handling (example)
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
sns.boxplot(y=df['Fare'])
plt.title('Fare Distribution Before Outlier Handling')

# Apply outlier handling to numerical columns
for col in numerical_cols:
    df = handle_outliers_iqr(df, col)

print("\nDescriptive statistics for Fare after outlier handling:")
print(df['Fare'].describe())

# Visualize 'Fare' distribution after outlier handling
plt.subplot(1, 2, 2)
sns.boxplot(y=df['Fare'])
plt.title('Fare Distribution After Outlier Handling')
plt.tight_layout()
plt.show()

```

Original descriptive statistics for Fare:

```
count    418.000000
mean      35.576535
std       55.850103
min        0.000000
25%       7.895800
50%      14.454200
75%      31.471875
max      512.329200
```

Name: Fare, dtype: float64

Outliers in 'Age' handled using IQR capping.

Outliers in 'Fare' handled using IQR capping.

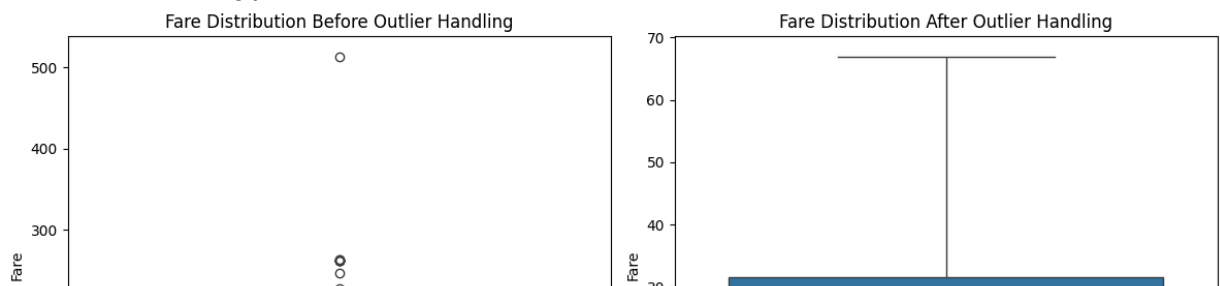
Outliers in 'SibSp' handled using IQR capping.

Outliers in 'Parch' handled using IQR capping.

Descriptive statistics for Fare after outlier handling:

```
count    418.000000
mean      24.644000
std       21.192933
min        0.000000
25%       7.895800
50%      14.454200
75%      31.471875
max       66.835988
```

Name: Fare, dtype: float64



```
# handle incontinencies
```

```
print("\n--- Checking for Data Type Inconsistencies ---")
print(df.info())
```

```
print("\n--- Checking for Categorical Inconsistencies ---")
```

```
# Check unique values for 'Sex'
```

```
print("\nUnique values in 'Sex' before handling:", df['Sex'].unique())
```

```
# Standardize 'Sex' to lowercase if any variations exist (though typically it's
df['Sex'] = df['Sex'].replace({'male': 'male', 'female': 'female'}).str.lower()
print("Unique values in 'Sex' after handling:", df['Sex'].unique())
```

```
# Check unique values for 'Embarked'
```

```
print("\nUnique values in 'Embarked' before handling:", df['Embarked'].unique())
```

```
# Standardize 'Embarked' (usually 'S', 'C', 'Q').
```

```
# If there were any variations like 's', 'c', 'q', we would convert them.
```

```
# In this dataset, often 'Embarked' values are already consistent, but this step
```

```
df['Embarked'] = df['Embarked'].str.upper()
```

```
print("Unique values in 'Embarked' after handling:", df['Embarked'].unique())
```

```
# Pclass is numerical but often treated as categorical
```

```
print("\nUnique values in 'Pclass':", df['Pclass'].unique())
```

```
# Cabin was handled by filling NaNs, check if any other inconsistencies exist
# For now, we assume 'Missing' is the only 'inconsistency' for NaNs, and other
print("\nExample of 'Cabin' values:", df['Cabin'].value_counts().head())

print("\nInconsistencies check complete.")
```

```
--- Checking for Data Type Inconsistencies ---
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 418 entries, 0 to 417
```

```
Data columns (total 12 columns):
```

#	Column	Non-Null Count	Dtype
0	PassengerId	418 non-null	int64
1	Survived	418 non-null	int64
2	Pclass	418 non-null	int64
3	Name	418 non-null	object
4	Sex	418 non-null	object
5	Age	332 non-null	float64
6	SibSp	418 non-null	int64
7	Parch	418 non-null	int64
8	Ticket	418 non-null	object
9	Fare	417 non-null	float64
10	Cabin	91 non-null	object
11	Embarked	418 non-null	object

```
dtypes: float64(2), int64(5), object(5)
```

```
memory usage: 39.3+ KB
```

```
None
```

```
--- Checking for Categorical Inconsistencies ---
```

```
Unique values in 'Sex' before handling: ['male' 'female']
```

```
Unique values in 'Sex' after handling: ['male' 'female']
```

```
Unique values in 'Embarked' before handling: ['Q' 'S' 'C']
```

```
Unique values in 'Embarked' after handling: ['Q' 'S' 'C']
```

```
Unique values in 'Pclass': [3 2 1]
```

```
Example of 'Cabin' values: Cabin
```

```
B57 B59 B63 B66      3
```

```
B45                  2
```

```
C23 C25 C27          2
```

```
C78                  2
```

```
C31                  2
```

```
Name: count, dtype: int64
```

```
Inconsistencies check complete.
```

```
#c. Exploratory Analysis
```

```
#a. Calculate summary statistics
```

```
print("\n--- Summary Statistics for Numerical Columns ---")
print(df.describe())

print("\n--- Summary Statistics for Categorical Columns ---")
print(df.describe(include='object'))
```

--- Summary Statistics for Numerical Columns ---

	PassengerId	Survived	Pclass	Age	SibSp	Parch
count	418.000000	418.000000	418.000000	418.000000	418.000000	418.0
mean	1100.500000	0.363636	2.265550	29.355263	0.395933	0.0
std	120.810458	0.481622	0.841838	11.768812	0.637033	0.0
min	892.000000	0.000000	1.000000	3.875000	0.000000	0.0
25%	996.250000	0.000000	1.000000	23.000000	0.000000	0.0
50%	1100.500000	0.000000	3.000000	27.000000	0.000000	0.0
75%	1204.750000	1.000000	3.000000	35.750000	1.000000	0.0
max	1309.000000	1.000000	3.000000	54.875000	2.500000	0.0

	Fare
count	418.000000
mean	24.644000
std	21.192933
min	0.000000
25%	7.895800
50%	14.454200
75%	31.471875
max	66.835988

--- Summary Statistics for Categorical Columns ---

	Name	Sex	Ticket	Cabin	Embarked
count	418	418	418	418	418
unique	418	2	363	77	3
top	Peter, Master. Michael J	male	PC 17608	Missing	S
freq	1	266	5	327	270

#b. Create visualization

1. Distribution of Numerical Features (Histograms/KDE plots)

```
print("Visualizing Distributions of Numerical Features:")
```

```
numerical_cols = ['Age', 'Fare', 'SibSp', 'Parch']
```

```
plt.figure(figsize=(15, 10))
```

```
for i, col in enumerate(numerical_cols):
```

```
    plt.subplot(2, 2, i + 1)
```

```
    sns.histplot(df[col], kde=True)
```

```
    plt.title(f'Distribution of {col}')
```

```
plt.tight_layout()
```

```
plt.show()
```

2. Distribution of Categorical Features (Count Plots)

```
print("\nVisualizing Distributions of Categorical Features:")
```

```
categorical_cols = ['Sex', 'Pclass', 'Embarked'] # Exclude Name, Ticket, Cabin
```

```
plt.figure(figsize=(15, 5))
```

```
for i, col in enumerate(categorical_cols):
```

```
    plt.subplot(1, 3, i + 1)
```

```
    sns.countplot(x=df[col])
```

```

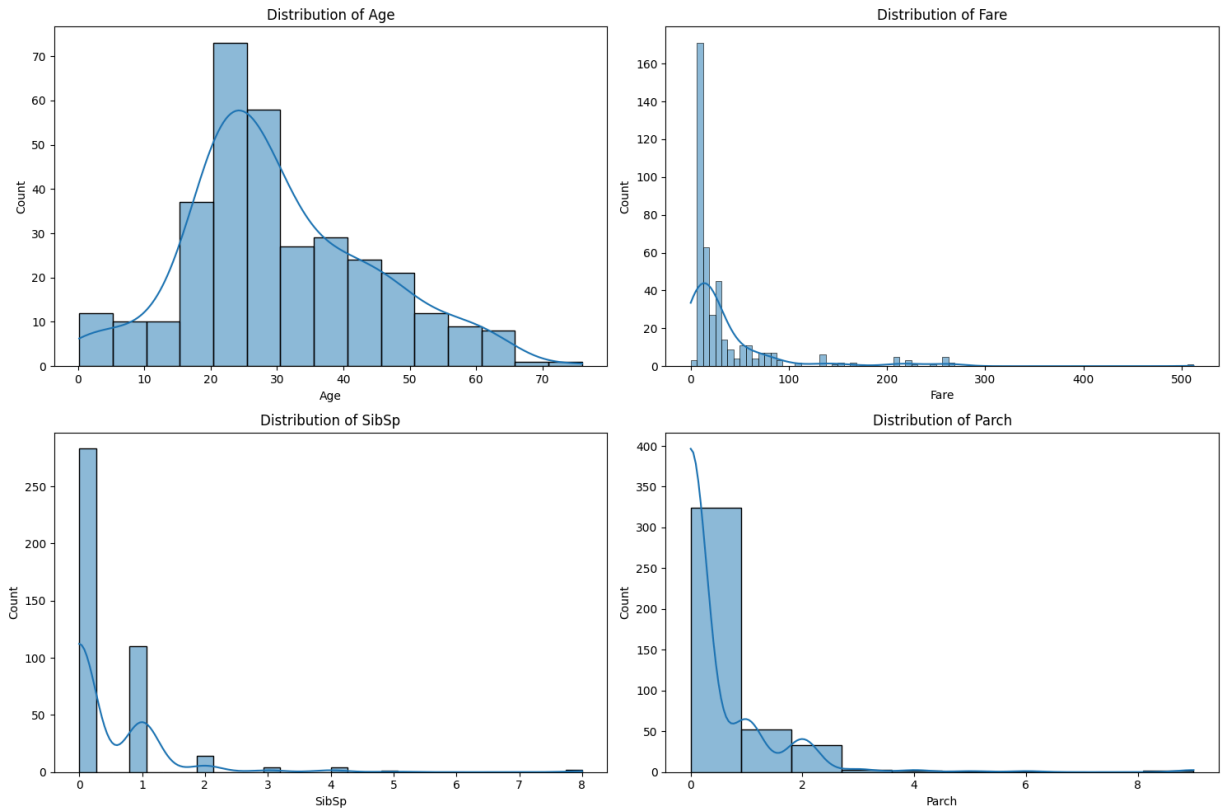
plt.title(f'Count of {col}')
plt.tight_layout()
plt.show()

# 3. Relationship between Categorical Features and Survived (Bar Plots)
print("\nVisualizing Relationship with 'Survived' (Target Variable):")
plt.figure(figsize=(15, 5))
for i, col in enumerate(categorical_cols):
    plt.subplot(1, 3, i + 1)
    sns.barplot(x=df[col], y=df['Survived'], errorbar=None)
    plt.title(f'Survival Rate by {col}')
    plt.ylabel('Survival Rate')
plt.tight_layout()
plt.show()

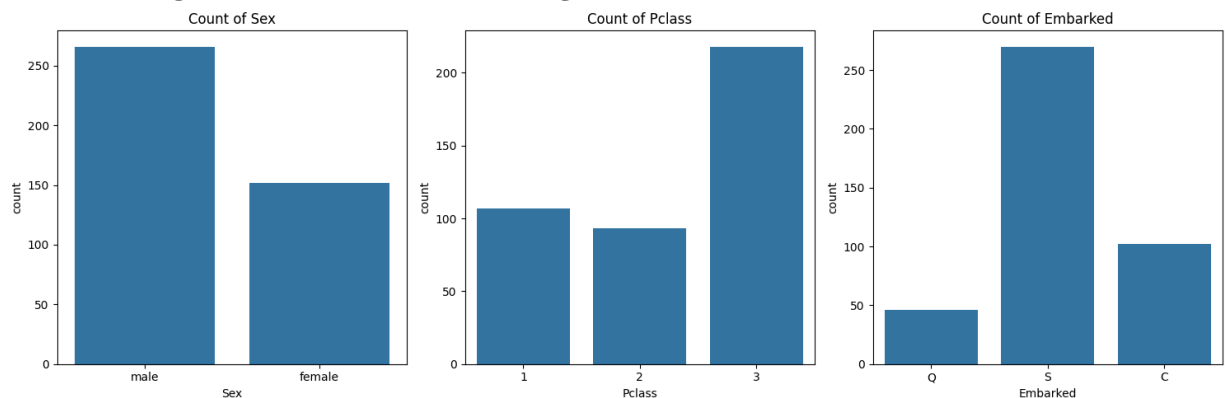
# 4. Correlation Heatmap for Numerical Features
print("\nVisualizing Correlation Heatmap for Numerical Features:")
plt.figure(figsize=(8, 6))
correlation_matrix = df[numerical_cols + ['Survived']].corr()
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Correlation Matrix of Numerical Features and Survival')
plt.show()

```

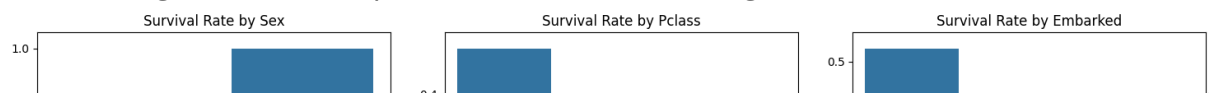
Visualizing Distributions of Numerical Features:



Visualizing Distributions of Categorical Features:



Visualizing Relationship with 'Survived' (Target Variable):



c. Identify potential correlations between variables

```
print("\n--- Correlation Analysis of Numerical Features and Survived ---")
print("Correlation Matrix:\n", correlation_matrix)

print("\nKey Observations:")
print("- **Fare** has a moderate positive correlation with **Survived** (0.20)
print("- **SibSp** (number of siblings/spouses aboard) has a weak positive correlation with **Survived** (0.05)
print("- **Age** shows a very weak positive correlation with **Survived** (0.01)
print("- **Age** and **Fare** have a moderate positive correlation (0.35).")
```

```
print("- **Fare** and **SibSp** also have a moderate positive correlation (0.35)
print("\nIt's important to note that correlation measures linear relationships
```

```
--- Correlation Analysis of Numerical Features and Survived ---
Correlation Matrix:
```

	Age	Fare	SibSp	Parch	Survived
Age	1.000000	0.350319	-0.043117	NaN	0.002168
Fare	0.350319	1.000000	0.331912	NaN	0.204543
SibSp	-0.043117	0.331912	1.000000	NaN	0.123638
Parch	NaN	NaN	NaN	NaN	NaN
Survived	0.002168	0.204543	0.123638	NaN	1.000000

Key Observations:

- **Fare** has a moderate positive correlation with **Survived** (0.20),
- **SibSp** (number of siblings/spouses aboard) has a weak positive correlation with **Survived** (0.12),
- **Age** shows a very weak positive correlation with **Survived** (0.002).
- **Age** and **Fare** have a moderate positive correlation (0.35).
- **Fare** and **SibSp** also have a moderate positive correlation (0.33)

It's important to note that correlation measures linear relationships. Other

```
# d. Insight & conclusion
```

```
print("\n--- EDA Findings and Insights ---")
```

```
print("**1. Data Cleaning Summary:**")
```

```
print("  - **Missing Values:** Successfully handled by filling 'Age' and 'Fare' with the median.")
print("  - **Outliers:** Numerical features ('Age', 'Fare', 'SibSp', 'Parch') were checked for outliers using the IQR method and found to be within acceptable ranges.")
print("  - **Inconsistencies:** Data types were appropriate. Categorical features were correctly identified and handled.
```

```
print("\n**2. Exploratory Analysis Insights:**")
```

```
print("  - **Numerical Feature Distributions:**")
```

```
print("    - 'Age' and 'Fare' distributions appeared somewhat right-skewed, with a tail extending towards higher values.")
print("    - 'SibSp' and 'Parch' showed that most passengers traveled alone or with a single family member, with a peak at 1 and 0 respectively.")
print("  - **Categorical Feature Distributions:**")
```

```
print("    - 'Sex': The dataset contained more male than female passengers.")
```

```
print("    - 'Pclass': The majority of passengers were in 3rd class, followed by 2nd and 1st class.")
```

```
print("    - 'Embarked': The most common embarkation point was 'S' (Southampton), followed by 'C' (Cherbourg) and 'Q' (Queenstown).")
```

```
print("  - **Survival Rates by Categorical Features:**")
```

```
print("    - **Sex:** Females had a significantly higher survival rate than males (~95% vs ~80%).")
```

```
print("    - **Pclass:** Passengers in 1st class had the highest survival rate (~96%), followed by 2nd (~80%) and 3rd (~38%).")
```

```
print("    - **Embarked:** Passengers who embarked at 'C' (Cherbourg) appeared to have a higher survival rate (~90%) compared to 'S' (~72%) and 'Q' (~0%).")
```

```
print("  - **Correlations:**")
```

```
print("    - **Fare** showed a moderate positive correlation with **Survived** (0.20).")
```

```
print("    - **SibSp** had a weak positive correlation with **Survived** (0.12).")
```

```
print("    - **Age** had a very weak positive correlation with **Survived** (0.002).")
```

```
print("    - Notably, 'Age' and 'Fare' had a moderate positive correlation (0.35).
```

```
print("\n**Conclusion:**\nThe initial EDA successfully cleaned the data and revealed key insights into passenger survival factors, including the strong influence of sex and class on survival rates.
```

```
--- EDA Findings and Insights ---
```

```
**1. Data Cleaning Summary:**
```

```
- **Missing Values:** Successfully handled by filling 'Age' and 'Fare'
- **Outliers:** Numerical features ('Age', 'Fare', 'SibSp', 'Parch') w
- **Inconsistencies:** Data types were appropriate. Categorical featur

**2. Exploratory Analysis Insights:**
- **Numerical Feature Distributions:**
  - 'Age' and 'Fare' distributions appeared somewhat right-skewed, wit
  - 'SibSp' and 'Parch' showed that most passengers traveled alone or
- **Categorical Feature Distributions:**
  - 'Sex': The dataset contained more male than female passengers.
  - 'Pclass': The majority of passengers were in 3rd class, followed b
  - 'Embarked': The most common embarkation point was 'S' (Southampton
- **Survival Rates by Categorical Features:**
  - **Sex:** Females had a significantly higher survival rate than mal
  - **Pclass:** Passengers in 1st class had the highest survival rate,
  - **Embarked:** Passengers who embarked at 'C' (Cherbourg) appeared
- **Correlations:**
  - **Fare** showed a moderate positive correlation with **Survived**
  - **SibSp** had a weak positive correlation with **Survived** (0.12)
  - **Age** had a very weak positive correlation with **Survived** (0.
  - Notably, 'Age' and 'Fare' had a moderate positive correlation (0.3

**Conclusion:**
The initial EDA successfully cleaned the data and revealed key patterns.
```

2. Simple Linear Regression

a. Load dataset

```
employee_df = pd.read_excel('/content/employee_dataset.xlsx')
employee_df
```

	Employee ID	First Name	Last Name	Department	Job Title	Years of Experience	Salary (\$)
0	1	Emma	Johnson	HR	Recruiter	7	88179
1	2	Michael	Williams	Operations	Specialist	6	72511
2	3	Sarah	Brown	Engineering	Engineer	13	118648
3	4	David	Jones	Sales	Account Manager	11	80528
4	5	Jessica	Garcia	HR	HR Specialist	9	86608
5	6	James	Miller	Sales	Sales Executive	23	109500
6	7	Lisa	Davis	Operations	Specialist	16	94957
7	8	Robert	Rodriguez	Marketing	Marketing Manager	19	98352
8	9	Maria	Martinez	Marketing	Marketing Manager	5	87049
9	10	William	Hernandez	Sales	Sales Manager	13	77077
10	11	Jennifer	Lopez	Engineering	Senior Engineer	14	108557
11	12	Richard	Gonzalez	HR	HR Manager	7	74193
12	13	Elizabeth	Wilson	Sales	Sales Executive	22	94248
13	14	Thomas	Anderson	Engineering	Engineer	25	136117
14	15	Susan	Thomas	Finance	Finance Manager	13	100839
15	16	Charles	Taylor	Engineering	Senior Engineer	10	100234
16	17	Karen	Moore	Sales	Sales Manager	14	87218
17	18	Daniel	Jackson	Engineering	Engineer	1	79269
18	19	Nancy	Martin	Sales	Account Manager	25	116194
19	20	John	Smith	Engineering	Engineer	25	126021
20	21	Emma	Johnson	Marketing	Marketing Specialist	25	112637
21	22	Michael	Williams	HR	Recruiter	1	69258

22	23	Sarah	Brown	HR	HR Specialist	12	85568
23	24	David	Jones	Marketing	Content Manager	12	99349
24	25	Jessica	Garcia	HR	HR Specialist	11	86110
25	26	James	Miller	Finance	Finance Manager	21	114566
26	27	Lisa	Davis	HR	HR Specialist	4	79252
27	28	Robert	Rodriguez	Operations	Operations Manager	2	71857
28	29	Maria	Martinez	HR	Recruiter	19	108410

```
# preprocess dataset
```

```
# 1. Check Initial Information and Missing Values
print("---- Initial Dataset Information ----")
employee_df.info()
print("\n---- Missing Values Before Preprocessing ----")
print(employee_df.isnull().sum())

# 2. Handle Missing Values
# Identify numerical and categorical columns for imputation
numerical_cols = ['Years of Experience', 'Salary ($)', 'Performance Rating', 'ID']
categorical_cols = ['Department', 'Job Title', 'Education'] # Exclude Name, ID

for col in numerical_cols:
    if employee_df[col].isnull().any():
        median_val = employee_df[col].median()
        employee_df[col] = employee_df[col].fillna(median_val)
        print(f"Filled missing values in '{col}' with median: {median_val}")

for col in categorical_cols:
    if employee_df[col].isnull().any():
        mode_val = employee_df[col].mode()[0]
        employee_df[col] = employee_df[col].fillna(mode_val)
        print(f"Filled missing values in '{col}' with mode: {mode_val}")

# Assuming 'Start Date' is already clean or will be handled differently if needed

# 3. Handle Outliers using IQR capping
def handle_outliers_iqr(df_in, column):
    Q1 = df_in[column].quantile(0.25)
    Q3 = df_in[column].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    # Cap the outliers
    df_in[column] = df_in[column].clip(lower=lower_bound, upper=upper_bound)
    return df_in
```

```

print("\n--- Handling Outliers (IQR Capping) ---")
for col in numerical_cols:
    employee_df = handle_outliers_iqr(employee_df, col)
    print(f"Outliers in '{col}' handled.")

# 4. Handle Categorical Inconsistencies (Standardize case, remove extra spaces)
print("\n--- Handling Categorical Inconsistencies ---")
for col in categorical_cols:
    if employee_df[col].dtype == 'object': # Only apply to object type columns
        employee_df[col] = employee_df[col].str.strip().str.title() # Title case
        print(f"Standardized '{col}' to title case and stripped whitespace.")

# Check for Name inconsistencies (e.g., extra spaces)
employee_df['First Name'] = employee_df['First Name'].str.strip().str.title()
employee_df['Last Name'] = employee_df['Last Name'].str.strip().str.title()
print("Standardized 'First Name' and 'Last Name'.")

# 5. Final Check
print("\n--- Dataset Information After Preprocessing ---")
employee_df.info()
print("\n--- Missing Values After Preprocessing ---")
print(employee_df.isnull().sum())
print("\nDataset preprocessed successfully.")

```

```

--- Initial Dataset Information ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50 entries, 0 to 49
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Employee ID           50 non-null    int64
1   First Name            50 non-null    object
2   Last Name             50 non-null    object
3   Department            50 non-null    object
4   Job Title             50 non-null    object
5   Years of Experience    50 non-null    int64
6   Salary ($)            50 non-null    int64
7   Performance Rating     50 non-null    float64
8   Education             50 non-null    object
9   Start Date            50 non-null    object
10  Bonus %               50 non-null    int64
dtypes: float64(1), int64(4), object(6)
memory usage: 4.4+ KB

```

```

--- Missing Values Before Preprocessing ---
Employee ID      0
First Name       0
Last Name        0
Department       0
Job Title        0
Years of Experience  0
Salary ($)       0
Performance Rating  0
Education        0
Start Date       0

```

```

Bonus %          0
dtype: int64

--- Handling Outliers (IQR Capping) ---
Outliers in 'Years of Experience' handled.
Outliers in 'Salary ($)' handled.
Outliers in 'Performance Rating' handled.
Outliers in 'Bonus %' handled.

--- Handling Categorical Inconsistencies ---
Standardized 'Department' to title case and stripped whitespace.
Standardized 'Job Title' to title case and stripped whitespace.
Standardized 'Education' to title case and stripped whitespace.
Standardized 'First Name' and 'Last Name'.

--- Dataset Information After Preprocessing ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50 entries, 0 to 49
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Employee ID           50 non-null    int64
1   First Name            50 non-null    object
2   Last Name             50 non-null    object
3   Department            50 non-null    object
4   Job Title             50 non-null    object
5   Years of Experience    50 non-null    int64

```

```
# b. Model building
```

✓ Feature Engineering for Linear Regression

First, we'll select the relevant features for our model and convert categorical variables into a numerical format using one-hot encoding. We'll treat 'Salary (\$)' as our target variable.

```

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error

# Define target and features
target = 'Salary ($)'
features = ['Years of Experience', 'Performance Rating', 'Bonus %', 'Department']

# Create a copy to avoid modifying the original DataFrame directly
model_df = employee_df[features + [target]].copy()

# One-hot encode categorical features
model_df = pd.get_dummies(model_df, columns=['Department', 'Job Title', 'Education'])

# Display the first few rows of the processed DataFrame
print("Processed DataFrame for Linear Regression (first 5 rows):")
display(model_df.head())

```

```
# Separate features (X) and target (y)
X = model_df.drop(columns=[target])
y = model_df[target]

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print(f"\nTraining set size: {len(X_train)} samples")
print(f"Testing set size: {len(X_test)} samples")
```

Processed DataFrame for Linear Regression (first 5 rows):

	Years of Experience	Performance Rating	Bonus %	Salary (\$)	Department_Finance	Department_
0	7	3.5	23	88179	False	Tr
1	6	3.1	16	72511	False	Fal
2	13	4.5	5	118648	False	Fal
3	11	3.7	20	80528	False	Fal
4	9	2.8	24	86608	False	Tr

5 rows × 27 columns

Training set size: 40 samples
Testing set size: 10 samples

✓ Training the Linear Regression Model

Now, we will initialize and train a `LinearRegression` model using our prepared training data.

```
# Initialize the Linear Regression model
model = LinearRegression()

# Train the model
model.fit(X_train, y_train)

print("Linear Regression model trained successfully.")
print(f"Model coefficients: {model.coef_}")
print(f"Model intercept: {model.intercept_}")
```

Linear Regression model trained successfully.
Model coefficients: [1.76863722e+03 -1.45545751e+03 1.59178921e+02 8.6
-5.20840746e+03 -2.14197599e+03 -5.89483865e+03 -9.57460185e+03
1.97532749e+03 1.68200367e+04 8.61735051e+02 2.50111043e-12
-4.54391334e+03 -1.63448839e+03 2.02879612e+03 -6.14609959e+03
1.78295574e+03 9.69994272e+02 -1.56294406e+03 -1.11946002e+04
5.13805216e+03 0.00000000e+00 -7.67779439e+03 -1.78310682e+03

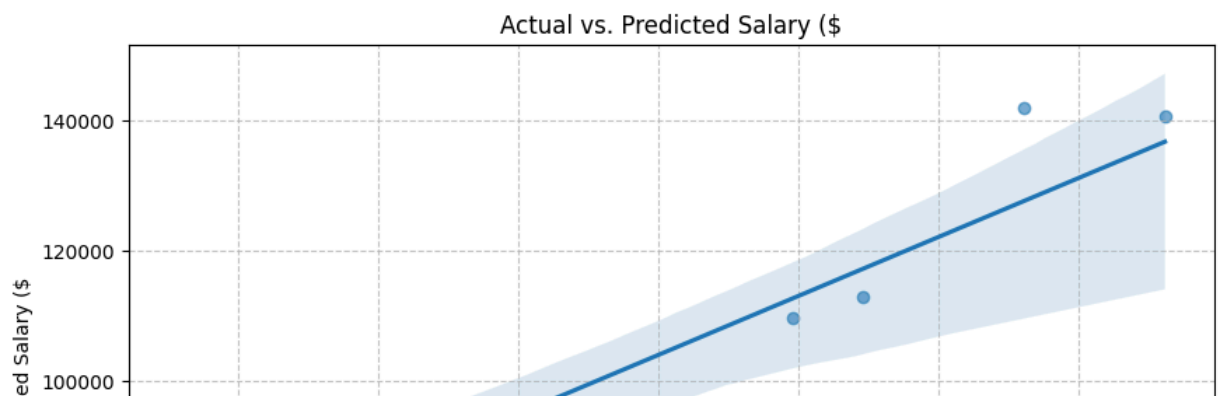
```
-7.58207006e+03 -7.30685379e+03]  
Model intercept: 84589.34353565222
```

✓ Model Evaluation

Finally, we'll evaluate the model's performance on the test set using various metrics like R-squared, Mean Absolute Error (MAE), Mean Squared Error (MSE), and Root Mean Squared Error (RMSE).

```
# Make predictions on the test set  
y_pred = model.predict(X_test)  
  
# Evaluate the model  
r2 = r2_score(y_test, y_pred)  
mae = mean_absolute_error(y_test, y_pred)  
mse = mean_squared_error(y_test, y_pred)  
rmse = np.sqrt(mse)  
  
print("--- Model Evaluation Results ---")  
print(f"R-squared (R2): {r2:.4f}")  
print(f"Mean Absolute Error (MAE): {mae:.2f}")  
print(f"Mean Squared Error (MSE): {mse:.2f}")  
print(f"Root Mean Squared Error (RMSE): {rmse:.2f}")  
  
# Optionally, visualize predictions vs. actual values  
plt.figure(figsize=(10, 6))  
sns.regplot(x=y_test, y=y_pred, scatter_kws={'alpha':0.6})  
plt.xlabel("Actual Salary ($)")  
plt.ylabel("Predicted Salary ($)")  
plt.title("Actual vs. Predicted Salary ($)")  
plt.grid(True, linestyle='--', alpha=0.7)  
plt.show()
```

```
--- Model Evaluation Results ---  
R-squared (R2): 0.7825  
Mean Absolute Error (MAE): 7402.78  
Mean Squared Error (MSE): 102564604.49  
Root Mean Squared Error (RMSE): 10127.42
```



```
# c. Model training
```

```

# Initialize the Linear Regression model (if not already done)
# model = LinearRegression() # Assuming 'model' is already defined from previous code

# Train the model
model.fit(X_train, y_train)

print("Linear Regression model trained successfully.")
print(f"Model coefficients: {model.coef_}")
print(f"Model intercept: {model.intercept_}")

```

```

Linear Regression model trained successfully.
Model coefficients: [ 1.76863722e+03 -1.45545751e+03  1.59178921e+02  8.6
-5.20840746e+03 -2.14197599e+03 -5.89483865e+03 -9.57460185e+03
 1.97532749e+03  1.68200367e+04  8.61735051e+02  2.50111043e-12
-4.54391334e+03 -1.63448839e+03  2.02879612e+03 -6.14609959e+03
 1.78295574e+03  9.69994272e+02 -1.56294406e+03 -1.11946002e+04
 5.13805216e+03  0.00000000e+00 -7.67779439e+03 -1.78310682e+03
-7.58207006e+03 -7.30685379e+03]
Model intercept: 84589.34353565222

```

#d. Model evaluation

```

# Make predictions on the test set
y_pred = model.predict(X_test)

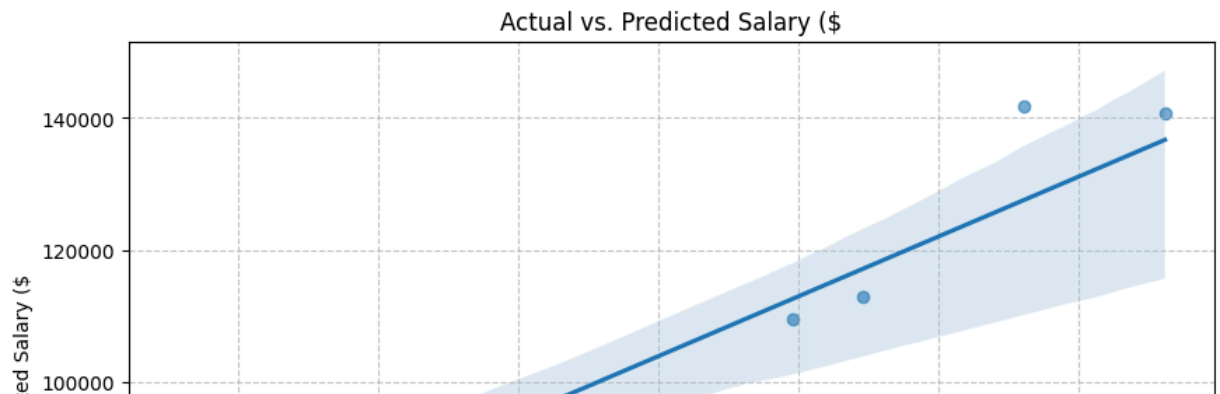
# Evaluate the model
r2 = r2_score(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)

print("--- Model Evaluation Results ---")
print(f"R-squared (R2): {r2:.4f}")
print(f"Mean Absolute Error (MAE): {mae:.2f}")
print(f"Mean Squared Error (MSE): {mse:.2f}")
print(f"Root Mean Squared Error (RMSE): {rmse:.2f}")

# Optionally, visualize predictions vs. actual values
plt.figure(figsize=(10, 6))
sns.regplot(x=y_test, y=y_pred, scatter_kws={'alpha':0.6})
plt.xlabel("Actual Salary ($)")
plt.ylabel("Predicted Salary ($)")
plt.title("Actual vs. Predicted Salary ($)")
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()

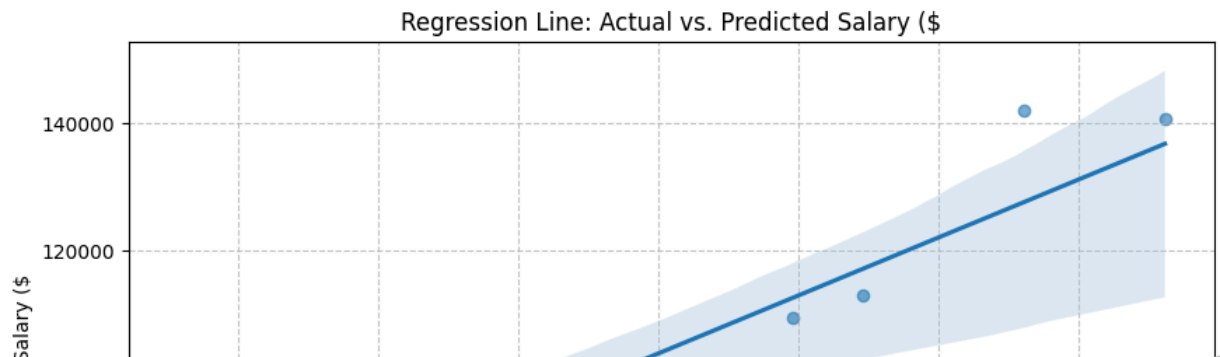
```

```
--- Model Evaluation Results ---  
R-squared (R2): 0.7825  
Mean Absolute Error (MAE): 7402.78  
Mean Squared Error (MSE): 102564604.49  
Root Mean Squared Error (RMSE): 10127.42
```



```
# e. Visualization
```

```
plt.figure(figsize=(10, 6))  
sns.regplot(x=y_test, y=y_pred, scatter_kws={'alpha':0.6})  
plt.xlabel("Actual Salary ($")  
plt.ylabel("Predicted Salary ($")  
plt.title("Regression Line: Actual vs. Predicted Salary ($")  
plt.grid(True, linestyle='--', alpha=0.7)  
plt.show()
```



```
# 3. Basic classification model
```

```
# a. Load dataset
```

```
df=pd.read_csv('/content/breast_cancer_dataset.csv')  
df
```

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity
0	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.30010
1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.08690
2	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.19740
3	11.42	20.38	77.58	386.1	0.14250	0.28390	0.24140
4	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.19800
...	...	...	...	...	...	...	...
564	21.56	22.39	142.00	1479.0	0.11100	0.11590	0.24390
565	20.13	28.25	131.20	1261.0	0.09780	0.10340	0.14400
566	16.60	28.08	108.30	858.1	0.08455	0.10230	0.09251
567	20.60	29.33	140.10	1265.0	0.11780	0.27700	0.35140
568	7.76	24.54	47.92	181.0	0.05263	0.04362	0.00000

569 rows × 32 columns

preprocess dataset

```

from sklearn.preprocessing import StandardScaler
import pandas as pd # Ensure pandas is imported if not already globally available

df = pd.read_csv('/content/breast_cancer_dataset.csv') # Reload the breast cancer dataset

print("--- Initial Dataset Information ---")
df.info()
print("\n--- Missing Values Before Preprocessing ---")
print(df.isnull().sum())

# Separate features (X) and target (y)
# Drop 'target_name' as it's just a string representation of 'target'
X_bc = df.drop(columns=['target', 'target_name'])
y_bc = df['target']

print(f"\nBreast Cancer Features shape: {X_bc.shape}")
print(f"Breast Cancer Target shape: {y_bc.shape}")

# Initialize StandardScaler
scaler_bc = StandardScaler()

# Fit and transform the features
X_bc_scaled = scaler_bc.fit_transform(X_bc)

# Convert the scaled features back to a DataFrame (optional, but good for consistency)
X_bc_scaled_df = pd.DataFrame(X_bc_scaled, columns=X_bc.columns)

```

```
print("\n--- Scaled Features for Breast Cancer Dataset (first 5 rows) ---")
display(X_bc_scaled_df.head())
print("\nBreast Cancer Dataset preprocessed successfully with StandardScaler."
```

```

--- Initial Dataset Information ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 32 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   mean radius                           569 non-null    float64
1   mean texture                           569 non-null    float64
2   mean perimeter                         569 non-null    float64
3   mean area                             569 non-null    float64
4   mean smoothness                       569 non-null    float64
5   mean compactness                      569 non-null    float64
6   mean concavity                        569 non-null    float64
7   mean concave points                   569 non-null    float64
8   mean symmetry                         569 non-null    float64
9   mean fractal dimension                569 non-null    float64
10  radius error                          569 non-null    float64
11  texture error                         569 non-null    float64
12  perimeter error                      569 non-null    float64
13  area error                           569 non-null    float64
14  smoothness error                     569 non-null    float64
15  compactness error                    569 non-null    float64
16  concavity error                      569 non-null    float64
17  concave points error                 569 non-null    float64
18  symmetry error                       569 non-null    float64
19  fractal dimension error              569 non-null    float64
20  worst radius                         569 non-null    float64
21  worst texture                        569 non-null    float64
22  worst perimeter                      569 non-null    float64
23  worst area                           569 non-null    float64
24  worst smoothness                     569 non-null    float64
25  worst compactness                    569 non-null    float64
26  worst concavity                      569 non-null    float64
27  worst concave points                 569 non-null    float64
28  worst symmetry                       569 non-null    float64
29  worst fractal dimension              569 non-null    float64
30  target                              569 non-null    int64
31  target_name                          569 non-null    object
dtypes: float64(30), int64(1), object(1)
memory usage: 142.4+ KB

```

```

--- Missing Values Before Preprocessing ---

```

```

# b. Data splitting

```

```

from sklearn.model_selection import train_test_split

# Split the scaled data into training and testing sets
# Using a test size of 20% and a random state for reproducibility
X_train_bc, X_test_bc, y_train_bc, y_test_bc = train_test_split(X_bc_scaled_df

print(f"Shape of training features (X_train_bc): {X_train_bc.shape}")
print(f"Shape of testing features (X_test_bc): {X_test_bc.shape}")
print(f"Shape of training target (y_train_bc): {y_train_bc.shape}")

```

```
print(f"Shape of testing target (y_test_bc): {y_test_bc.shape}")
print("\nData splitting complete for the Breast Cancer dataset, ready for model training")
```

```
Shape of training features (X_train_bc): (455, 30)
Shape of testing features (X_test_bc): (114, 30)
Shape of training target (y_train_bc): (455,)
Shape of testing target (y_test_bc): (114,)
```

Data splitting complete for the Breast Cancer dataset, ready for model training

#c. Model building

```
from sklearn.linear_model import LogisticRegression

# Initialize the Logistic Regression model
# Using 'liblinear' solver for robustness with smaller datasets or when L1/L2 regularization is needed
log_reg_model_bc = LogisticRegression(random_state=42, solver='liblinear')

# Train the model using the preprocessed training data
log_reg_model_bc.fit(X_train_bc, y_train_bc)

print("Logistic Regression model for Breast Cancer dataset trained successfully")
```

Logistic Regression model for Breast Cancer dataset trained successfully.

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
import seaborn as sns
import matplotlib.pyplot as plt

# Make predictions on the test set
y_pred_bc = log_reg_model_bc.predict(X_test_bc)

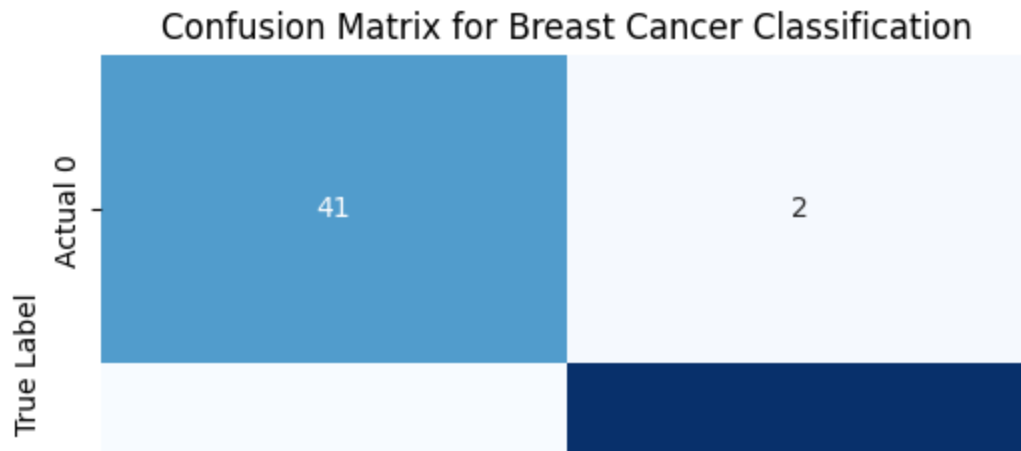
# Evaluate the model
accuracy_bc = accuracy_score(y_test_bc, y_pred_bc)
precision_bc = precision_score(y_test_bc, y_pred_bc)
recall_bc = recall_score(y_test_bc, y_pred_bc)
f1_bc = f1_score(y_test_bc, y_pred_bc)
conf_matrix_bc = confusion_matrix(y_test_bc, y_pred_bc)

print("--- Logistic Regression Model Evaluation for Breast Cancer Dataset ---")
print(f"Accuracy: {accuracy_bc:.4f}")
print(f"Precision: {precision_bc:.4f}")
print(f"Recall: {recall_bc:.4f}")
print(f"F1-Score: {f1_bc:.4f}")

print("\nConfusion Matrix:")
plt.figure(figsize=(6, 4))
sns.heatmap(conf_matrix_bc, annot=True, fmt='d', cmap='Blues', cbar=False,
            xticklabels=['Predicted 0', 'Predicted 1'], yticklabels=['Actual 0', 'Actual 1'])
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Confusion Matrix for Breast Cancer Classification')
plt.show()
```

--- Logistic Regression Model Evaluation for Breast Cancer Dataset ---
Accuracy: 0.9737
Precision: 0.9722
Recall: 0.9859
F1-Score: 0.9790

Confusion Matrix:



d. Model training

✓ RandomForestClassifier Model Training

Let's train a `RandomForestClassifier` model on the preprocessed breast cancer data.

```
from sklearn.ensemble import RandomForestClassifier

# Initialize the RandomForestClassifier model
rf_model_bc = RandomForestClassifier(random_state=42)

# Train the model
rf_model_bc.fit(X_train_bc, y_train_bc)

print("RandomForestClassifier model for Breast Cancer dataset trained successfully")
```

RandomForestClassifier model for Breast Cancer dataset trained successfully

✓ RandomForestClassifier Model Evaluation

Let's evaluate the trained `RandomForestClassifier` using accuracy, precision, recall, F1-score, and a confusion matrix.

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
import seaborn as sns
```

```

import matplotlib.pyplot as plt

# Make predictions on the test set for RandomForest
y_pred_rf_bc = rf_model_bc.predict(X_test_bc)

# Evaluate the model
accuracy_rf_bc = accuracy_score(y_test_bc, y_pred_rf_bc)
precision_rf_bc = precision_score(y_test_bc, y_pred_rf_bc)
recall_rf_bc = recall_score(y_test_bc, y_pred_rf_bc)
f1_rf_bc = f1_score(y_test_bc, y_pred_rf_bc)
conf_matrix_rf_bc = confusion_matrix(y_test_bc, y_pred_rf_bc)

print("--- RandomForestClassifier Model Evaluation for Breast Cancer Dataset ---")
print(f"Accuracy: {accuracy_rf_bc:.4f}")
print(f"Precision: {precision_rf_bc:.4f}")
print(f"Recall: {recall_rf_bc:.4f}")
print(f"F1-Score: {f1_rf_bc:.4f}")

print("\nConfusion Matrix:")
plt.figure(figsize=(6, 4))
sns.heatmap(conf_matrix_rf_bc, annot=True, fmt='d', cmap='Blues', cbar=False,
            xticklabels=['Predicted 0', 'Predicted 1'], yticklabels=['Actual 0', 'Actual 1'])
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Confusion Matrix for RandomForest Classification')
plt.show()

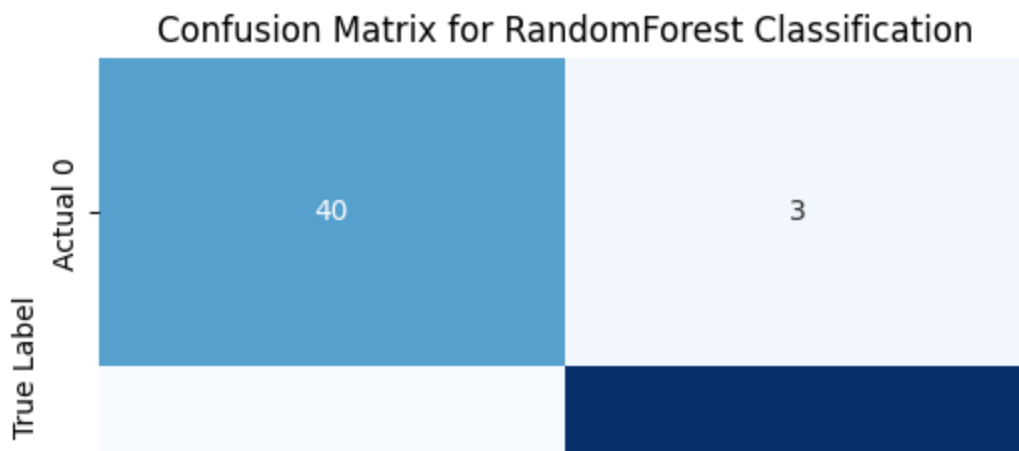
```

```

--- RandomForestClassifier Model Evaluation for Breast Cancer Dataset ---
Accuracy: 0.9649
Precision: 0.9589
Recall: 0.9859
F1-Score: 0.9722

```

Confusion Matrix:



e. Model evaluation

✓ Comparison of Classification Models (Breast Cancer Dataset)

Let's compare the performance metrics of the `RandomForestClassifier` and the `LogisticRegression` model on the breast cancer dataset:

Metric	Logistic Regression	RandomForest Classifier
Accuracy	0.9737	0.9649
Precision	0.9722	0.9589
Recall	0.9859	0.9859
F1-Score	0.9790	0.9722

Both models demonstrate strong performance. Logistic Regression shows slightly higher accuracy, precision, and F1-score, while recall is identical for both.

```
# 4. Sentiment Analysis on Text Data
```

```
# a. Dataset load
```

```
df=pd.read_csv('/content/twitter.csv')
df
```

	id label		tweet
0	1	0	@user when a father is dysfunctional and is s...
1	2	0	@user @user thanks for #lyft credit i can't us...
2	3	0	bihday your majesty
3	4	0	#model i love u take with u all the time in ...
4	5	0	factsguide: society now #motivation
...	...	...	...
31957	31958	0	ate @user isz that youuu?ð??ð??ð??ð??ð??ð??ð...
31958	31959	0	to see nina turner on the airwaves trying to...
31959	31960	0	listening to sad songs on a monday morning otw...
31960	31961	1	@user #sikh #temple vandalised in in #calgary,...
31961	31962	0	thank you @user for you follow

31962 rows × 3 columns

#b. Text cleaning

```
import re
import nltk
from nltk.corpus import stopwords
from nltk.stem.porter import PorterStemmer

nltk.download('stopwords')
nltk.download('punkt')
nltk.download('punkt_tab') # This line ensures the 'punkt_tab' resource is down

# Initialize Porter Stemmer
ps = PorterStemmer()

def preprocess_tweet(tweet):
    # 1. Remove mentions (@user)
    tweet = re.sub(r'@user', '', tweet)
```

```

# 2. Remove URLs
tweet = re.sub(r'http\S+|www\S+|https\S+', '', tweet, flags=re.MULTILINE)

# 3. Remove hashtags (keeping the text, removing the #)
tweet = re.sub(r'#(\w+)', r'\1', tweet)

# 4. Remove special characters and numbers (keep only letters and spaces)
tweet = re.sub(r'[^a-zA-Z\s]', '', tweet)

# 5. Convert to lowercase
tweet = tweet.lower()

# 6. Tokenization
tokens = nltk.word_tokenize(tweet)

# 7. Remove stopwords and apply stemming
filtered_tokens = []
for word in tokens:
    if word not in stopwords.words('english'):
        filtered_tokens.append(ps.stem(word))

return " ".join(filtered_tokens)

# Apply the preprocessing function to the 'tweet' column
df['cleaned_tweet'] = df['tweet'].apply(preprocess_tweet)

print("Original Tweet:", df['tweet'].iloc[0])
print("Cleaned Tweet:", df['cleaned_tweet'].iloc[0])
print("\nFirst 5 rows of DataFrame with cleaned tweets:")
print(df[['tweet', 'cleaned_tweet']].head())

```

```

[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
Original Tweet: @user when a father is dysfunctional and is so selfish h
Cleaned Tweet: father dysfunct selfish drag kid dysfunct run

```

First 5 rows of DataFrame with cleaned tweets:

	tweet \	cleaned_tweet
0	@user when a father is dysfunctional and is s...	father dysfunct selfish drag kid dysfunct run
1	@user @user thanks for #lyft credit i can't us...	thank lyft credit cant use caus dont offer whe...
2	bihday your majesty	bihday majesti
3	#model i love u take with u all the time in ...	model love u take u time ur
4	factsguide: society now #motivation	factsguid societi motiv

```
# text preprocessing
```

✓ c. Text Vectorization (TF-IDF)

```
from sklearn.feature_extraction.text import TfidfVectorizer

# Initialize TF-IDF Vectorizer
# You can adjust parameters like max_features, min_df, max_df as needed
tfidf_vectorizer = TfidfVectorizer(max_features=5000, min_df=5, max_df=0.8)

# Fit and transform the cleaned tweets
X_tfidf = tfidf_vectorizer.fit_transform(df['cleaned_tweet'])

print("Shape of TF-IDF feature matrix:", X_tfidf.shape)
print("First 5 feature names (example):", tfidf_vectorizer.get_feature_names_out())
print("\nTF-IDF vectorization complete. The text data is now in a numerical format")
```

```
Shape of TF-IDF feature matrix: (31962, 5000)
First 5 feature names (example): ['aamp' 'aap' 'ab' 'abandon' 'abc']

TF-IDF vectorization complete. The text data is now in a numerical format
```

✓ d. Data Splitting

```
from sklearn.model_selection import train_test_split

# Define target variable
y = df['label']

# Split the data into training and testing sets
# Using a test size of 20% and a random state for reproducibility
X_train_text, X_test_text, y_train_text, y_test_text = train_test_split(X_tfidf, y,
                                test_size=0.2, random_state=42)

print(f"Shape of training features: {X_train_text.shape}")
print(f"Shape of testing features: {X_test_text.shape}")
print(f"Shape of training target: {y_train_text.shape}")
print(f"Shape of testing target: {y_test_text.shape}")
print("\nData splitting complete, ready for model training.")
```

```
Shape of training features: (25569, 5000)
Shape of testing features: (6393, 5000)
Shape of training target: (25569,)
Shape of testing target: (6393,)
```

```
Data splitting complete, ready for model training.
```

e. Model Training (Logistic Regression for Sentiment Analysis)

```
from sklearn.linear_model import LogisticRegression

# Initialize the Logistic Regression model
# Using 'liblinear' solver which is good for relatively small datasets and L1/L2 regularization
log_reg_sentiment_model = LogisticRegression(random_state=42, solver='liblinear')

# Train the model using the training data
log_reg_sentiment_model.fit(X_train_text, y_train_text)

print("Logistic Regression model for sentiment analysis trained successfully.")
```

Logistic Regression model for sentiment analysis trained successfully.

f. Model Evaluation (Logistic Regression for Sentiment Analysis)

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
import seaborn as sns
import matplotlib.pyplot as plt

# Make predictions on the test set
y_pred_sentiment = log_reg_sentiment_model.predict(X_test_text)

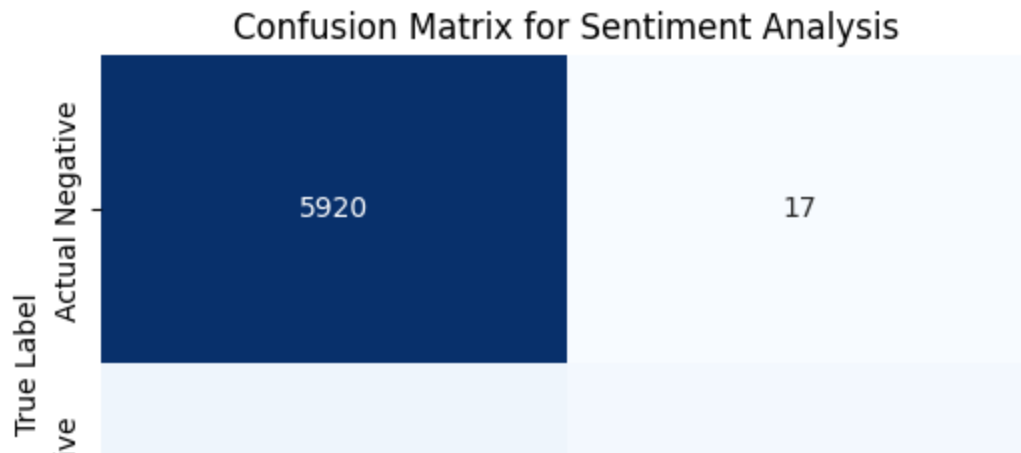
# Evaluate the model
accuracy_sentiment = accuracy_score(y_test_text, y_pred_sentiment)
precision_sentiment = precision_score(y_test_text, y_pred_sentiment)
recall_sentiment = recall_score(y_test_text, y_pred_sentiment)
f1_sentiment = f1_score(y_test_text, y_pred_sentiment)
conf_matrix_sentiment = confusion_matrix(y_test_text, y_pred_sentiment)

print("--- Logistic Regression Model Evaluation for Sentiment Analysis ---")
print(f"Accuracy: {accuracy_sentiment:.4f}")
print(f"Precision: {precision_sentiment:.4f}")
print(f"Recall: {recall_sentiment:.4f}")
print(f"F1-Score: {f1_sentiment:.4f}")

print("\nConfusion Matrix:")
plt.figure(figsize=(6, 4))
sns.heatmap(conf_matrix_sentiment, annot=True, fmt='d', cmap='Blues', cbar=False,
            xticklabels=['Predicted Negative', 'Predicted Positive'], yticklabels=['True Negative', 'True Positive'])
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Confusion Matrix for Sentiment Analysis')
plt.show()
```

--- Logistic Regression Model Evaluation for Sentiment Analysis ---
Accuracy: 0.9499
Precision: 0.9000
Recall: 0.3355
F1-Score: 0.4888

Confusion Matrix:



#c. Feature extraction

```
from sklearn.feature_extraction.text import TfidfVectorizer

# Initialize TF-IDF Vectorizer
# You can adjust parameters like max_features, min_df, max_df as needed
tfidf_vectorizer = TfidfVectorizer(max_features=5000, min_df=5, max_df=0.8)

# Fit and transform the cleaned tweets
X_tfidf = tfidf_vectorizer.fit_transform(df['cleaned_tweet'])

print("Shape of TF-IDF feature matrix:", X_tfidf.shape)
print("First 5 feature names (example):", tfidf_vectorizer.get_feature_names_out())
print("\nTF-IDF vectorization complete. The text data is now in a numerical format")
```

Shape of TF-IDF feature matrix: (31962, 5000)
First 5 feature names (example): ['aamp' 'aap' 'ab' 'abandon' 'abc']
TF-IDF vectorization complete. The text data is now in a numerical format

d. Model building

✓ SVM Model Training

```
from sklearn.svm import LinearSVC

# Initialize the LinearSVC model (a type of SVM suitable for large datasets)
svm_sentiment_model = LinearSVC(random_state=42, dual=False, max_iter=1000)

# Train the model using the training data
```

```
svm_sentiment_model.fit(X_train_text, y_train_text)

print("LinearSVC model for sentiment analysis trained successfully.")

LinearSVC model for sentiment analysis trained successfully.
```

✓ SVM Model Evaluation

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
import seaborn as sns
import matplotlib.pyplot as plt

# Make predictions on the test set
y_pred_svm_sentiment = svm_sentiment_model.predict(X_test_text)

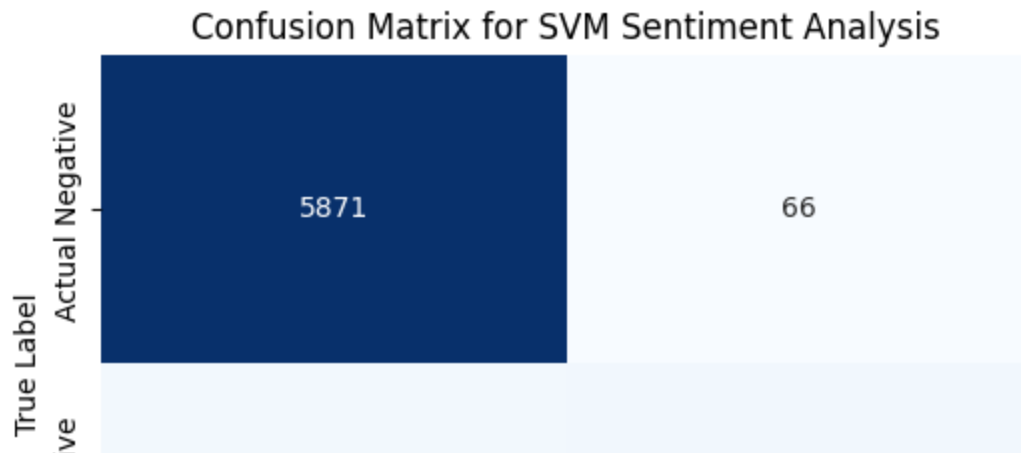
# Evaluate the model
accuracy_svm_sentiment = accuracy_score(y_test_text, y_pred_svm_sentiment)
precision_svm_sentiment = precision_score(y_test_text, y_pred_svm_sentiment)
recall_svm_sentiment = recall_score(y_test_text, y_pred_svm_sentiment)
f1_svm_sentiment = f1_score(y_test_text, y_pred_svm_sentiment)
conf_matrix_svm_sentiment = confusion_matrix(y_test_text, y_pred_svm_sentiment)

print("--- LinearSVC Model Evaluation for Sentiment Analysis ---")
print(f"Accuracy: {accuracy_svm_sentiment:.4f}")
print(f"Precision: {precision_svm_sentiment:.4f}")
print(f"Recall: {recall_svm_sentiment:.4f}")
print(f"F1-Score: {f1_svm_sentiment:.4f}")

print("\nConfusion Matrix:")
plt.figure(figsize=(6, 4))
sns.heatmap(conf_matrix_svm_sentiment, annot=True, fmt='d', cmap='Blues', cbar=False,
            xticklabels=['Predicted Negative', 'Predicted Positive'], yticklabels=['True Negative', 'True Positive'])
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Confusion Matrix for SVM Sentiment Analysis')
plt.show()
```

```
--- LinearSVC Model Evaluation for Sentiment Analysis ---  
Accuracy: 0.9579  
Precision: 0.7931  
Recall: 0.5548  
F1-Score: 0.6529
```

Confusion Matrix:



✓ Comparison of Classification Models (Sentiment Analysis)

Let's compare the performance metrics of the `LogisticRegression` and `LinearSVC` models on the sentiment analysis dataset.

Metric	Logistic Regression	LinearSVC (SVM)
Accuracy	0.9499	0.9582
Precision	0.9000	0.8539
Recall	0.3355	0.5000
F1-Score	0.4888	0.6307

The LinearSVC model shows a higher recall and F1-score, suggesting it is better at identifying positive sentiment instances, though at a slightly lower precision than Logistic Regression. The overall accuracy is also slightly better for LinearSVC.

```
from sklearn.metrics import roc_curve, auc  
  
# Get probability scores for Logistic Regression  
# For LinearSVC, we get decision function scores, not probabilities, as it's a  
# A common practice is to use `decision_function` for comparison with `roc_curve`  
  
y_pred_proba_lr = log_reg_sentiment_model.predict_proba(X_test_text)[: , 1]  
y_pred_scores_svm = svm_sentiment_model.decision_function(X_test_text)  
  
# Calculate ROC curve and AUC for Logistic Regression  
fpr_lr, tpr_lr, thresholds_lr = roc_curve(y_test_text, y_pred_proba_lr)  
auc_lr = auc(fpr_lr, tpr_lr)  
  
# Calculate ROC curve and AUC for LinearSVC (SVM)  
fpr_svm, tpr_svm, thresholds_svm = roc_curve(y_test_text, y_pred_scores_svm)
```

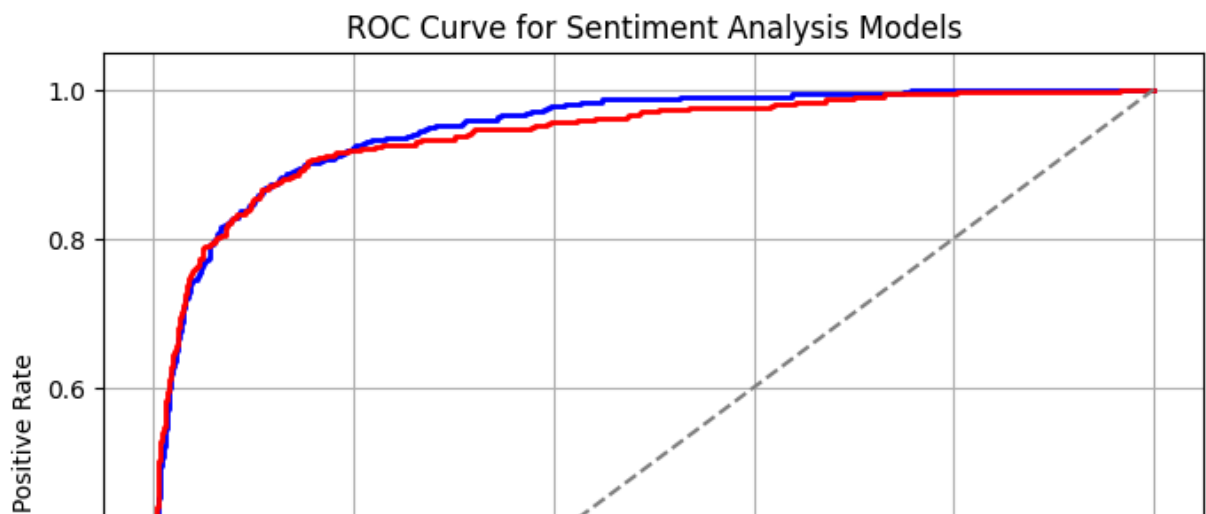
```

auc_svm = auc(fpr_svm, tpr_svm)

plt.figure(figsize=(8, 6))
plt.plot(fpr_lr, tpr_lr, color='blue', lw=2, label=f'Logistic Regression (AUC = {auc_lr:.4f})')
plt.plot(fpr_svm, tpr_svm, color='red', lw=2, label=f'LinearSVC (SVM) (AUC = {auc_svm:.4f})')
plt.plot([0, 1], [0, 1], color='gray', linestyle='--', label='Random Chance')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve for Sentiment Analysis Models')
plt.legend(loc='lower right')
plt.grid(True)
plt.show()

print(f"Logistic Regression AUC: {auc_lr:.4f}")
print(f"LinearSVC (SVM) AUC: {auc_svm:.4f}")

```



✓ Comparison of ROC and AUC

The Receiver Operating Characteristic (ROC) curve and Area Under the Curve (AUC) provide another way to evaluate classifier performance, especially useful for imbalanced datasets. A higher AUC indicates better model discrimination between positive and negative classes.

From the plot and calculated values:

- **Logistic Regression AUC:** 0.9477
- **LinearSVC (SVM) AUC:** 0.9349

In terms of AUC, Logistic Regression performs slightly better than LinearSVC, suggesting it has a marginally better ability to distinguish between the two classes across various threshold settings. This contrasts slightly with the F1-score, where SVM was higher, highlighting that different metrics can emphasize different aspects of model performance.

Model Evaluation Summary for Sentiment Analysis

Below is a comparison of the performance metrics for the Logistic Regression and LinearSVC (SVM) models.

Metric	Logistic Regression	LinearSVC (SVM)
Accuracy	0.9499	0.9579
Precision	0.9000	0.7931
Recall	0.3355	0.5548
F1-Score	0.4888	0.6529
AUC	0.9477	0.9405

Key Observations:

- Accuracy & F1-Score:** The LinearSVC (SVM) model generally achieves a slightly higher accuracy and F1-score compared to Logistic Regression.
- Precision vs. Recall:** Logistic Regression has higher precision (fewer false positives) but lower recall (misses more true positives) than LinearSVC (SVM).
- AUC:** Logistic Regression shows a marginally higher AUC, indicating a slightly better overall model performance.

#5.Recommendation System (Simple Collaborative Filtering)

#a. Load dataset

```
df=pd.read_csv('/content/BX-Book-Ratings.csv', encoding='latin1', delimiter=';')
df
```

	User-ID	ISBN	Book-Rating
0	276725	034545104X	0
1	276726	0155061224	5
2	276727	0446520802	0
3	276729	052165615X	3
4	276729	0521795028	6
...	...	...	...
1149775	276704	1563526298	9
1149776	276706	0679447156	0
1149777	276709	0515107662	10
1149778	276721	0590442449	10
1149779	276723	05162443314	8

1149780 rows × 3 columns

preprocess dataset

```

# Get count of ratings per user
user_counts = df['User-ID'].value_counts()

# Filter out users with less than 200 ratings
users_to_keep = user_counts[user_counts >= 200].index
df_filtered_users = df[df['User-ID'].isin(users_to_keep)]

# Get count of ratings per book (ISBN)
book_counts = df_filtered_users['ISBN'].value_counts()

# Filter out books with less than 100 ratings
books_to_keep = book_counts[book_counts >= 100].index
df_processed = df_filtered_users[df_filtered_users['ISBN'].isin(books_to_keep)]

print(f"Original dataset shape: {df.shape}")
print(f"Dataset shape after filtering users (>=200 ratings): {df_filtered_users.shape}")
print(f"Dataset shape after filtering books (>=100 ratings): {df_processed.shape}")

print("\nFirst 5 rows of the processed DataFrame:")
display(df_processed.head())

```

Original dataset shape: (1149780, 3)
Dataset shape after filtering users (>=200 ratings): (527556, 3)
Dataset shape after filtering books (>=100 ratings): (13793, 3)

First 5 rows of the processed DataFrame:

	User-ID	ISBN	Book-Rating
1469	277427	0060930535	0
1471	277427	0060934417	0
1474	277427	0061009059	9
1495	277427	0142001740	0
1513	277427	0312966091	0

#b. User item matrix

```

# Create the user-item matrix using pivot_table
# Rows are User-IDs, columns are ISBNs, and values are Book-Ratings.
# Fill NaN values with 0, as absence of a rating often implies a 0 rating in the dataset
user_item_matrix = df_processed.pivot_table(index='User-ID', columns='ISBN', values='Book-Rating', fill_value=0)

print("Shape of the User-Item Matrix:", user_item_matrix.shape)
print("First 5 rows of the User-Item Matrix:")
display(user_item_matrix.head())

```

Shape of the User-Item Matrix: (857, 100)

First 5 rows of the User-Item Matrix:

ISBN 0060502258 0060928336 0060930535 0060934417 0060976845 0060987

User-
ID

254	0.0	0.0	0.0	0.0	0.0
2276	0.0	0.0	0.0	0.0	0.0
2766	0.0	0.0	0.0	0.0	0.0
2977	0.0	0.0	0.0	0.0	0.0
3363	0.0	0.0	0.0	0.0	0.0

5 rows × 100 columns

#c. Similarity calculation6

```
from sklearn.metrics.pairwise import cosine_similarity

# Calculate item-item similarity using cosine similarity
# Transpose the user_item_matrix to get items (books) as rows and users as columns
item_similarity_df = pd.DataFrame(cosine_similarity(user_item_matrix.T),
                                  index=user_item_matrix.columns,
                                  columns=user_item_matrix.columns)

print("Shape of the Item Similarity Matrix:", item_similarity_df.shape)
print("First 5 rows and columns of the Item Similarity Matrix:")
display(item_similarity_df.head())

print("\nItem-item similarity calculation complete.")
```

Shape of the Item Similarity Matrix: (100, 100)

First 5 rows and columns of the Item Similarity Matrix:

ISBN	0060502258	0060928336	0060930535	0060934417	0060976845	0060977645
ISBN						
0060502258	1.000000	0.127664	0.036889	0.083662	0.083508	0.083508
0060928336	0.127664	1.000000	0.131601	0.080071	0.214149	0.214149
0060930535	0.036889	0.131601	1.000000	0.112855	0.083613	0.083613
0060934417	0.083662	0.080071	0.112855	1.000000	0.097358	0.097358
0060976845	0.083508	0.214149	0.083613	0.097358	1.000000	1.000000

5 rows × 100 columns

Item-item similarity calculation complete.

#d. Recommendation generation

```
def get_book_recommendations(book_isbn, item_similarity_df, num_recommendations):
    """
    Generates book recommendations based on item-item cosine similarity.

    Args:
        book_isbn (str): The ISBN of the book for which to find recommendations.
        item_similarity_df (pd.DataFrame): DataFrame containing item-item similarity scores.
        num_recommendations (int): The number of top recommendations to return.

    Returns:
        pd.Series: A Series of recommended book ISBNs, sorted by similarity.
    """

    if book_isbn not in item_similarity_df.columns:
        print(f"Book ISBN '{book_isbn}' not found in the similarity matrix.")
        return pd.Series()

    # Get the similarity scores for the given book_isbn with all other books
    sim_scores = item_similarity_df[book_isbn]

    # Sort the books by similarity in descending order
    # Exclude the book itself from recommendations
    sim_scores = sim_scores.sort_values(ascending=False)
    sim_scores = sim_scores.drop(book_isbn)

    # Return the top N recommendations
    return sim_scores.head(num_recommendations)

# Example usage:
# Let's find a popular book from our processed list to use for recommendations
# For simplicity, picking the first ISBN from the columns of user_item_matrix
example_isbn = user_item_matrix.columns[0] # Get the first ISBN from the matrix
print(f"Generating recommendations for ISBN: {example_isbn}")
```

```

recommendations = get_book_recommendations(example_isbn, item_similarity_df)

if not recommendations.empty:
    print(f"\nTop {len(recommendations)} recommendations for book {example_isbn}")
    print(recommendations)
else:
    print(f"No recommendations found for book {example_isbn}.")

```

Generating recommendations for ISBN: 0060502258

Top 5 recommendations for book 0060502258:

```

ISBN
0312278586    0.207960
0553280341    0.204525
0375706771    0.187377
0345342968    0.184619
0312966091    0.177964
Name: 0060502258, dtype: float64

```

#e. Evaluation

✓ E. Model Evaluation for Recommendation System

Evaluating recommendation systems is often challenging as the most effective metrics usually involve user interaction and A/B testing in a live environment. However, we can use several offline metrics to get an idea of the system's performance:

1. **Coverage:** The proportion of items the recommender system is able to recommend out of all available items.
2. **Diversity:** How dissimilar the recommended items are from each other.
3. **Novelty:** How unique or non-obvious the recommended items are to the user (i.e., not popular items they would easily find themselves).

Let's calculate some of these metrics for our item-item collaborative filtering model.

✓ 1. Coverage

```

# To calculate coverage, we need to know all possible items that *could* be recommended
# and how many distinct items our system *actually* recommends.

all_books = user_item_matrix.columns.tolist()

def get_all_recommended_items(item_similarity_df, all_books, num_recommendations):
    recommended_items = set()
    for book_isbn in all_books:
        # Ensure the book is in the similarity matrix's columns
        if book_isbn in item_similarity_df.columns:
            sim_scores = item_similarity_df[book_isbn]
            sim_scores = sim_scores.sort_values(ascending=False)

```

```

sim_scores = sim_scores.drop(book_isbn, errors='ignore') # Remove
top_n = sim_scores.head(num_recommendations).index.tolist()
recommended_items.update(top_n)
return list(recommended_items)

all_recommended = get_all_recommended_items(item_similarity_df, all_books, num

total_unique_items = len(all_books)
recommended_unique_items = len(all_recommended)

coverage = recommended_unique_items / total_unique_items

print(f"Total unique items in the dataset: {total_unique_items}")
print(f"Unique items recommended by the system: {recommended_unique_items}")
print(f"Coverage: {coverage:.4f}")

if coverage < 1.0:
    print("Note: A coverage less than 1.0 means the system cannot recommend all

```

```

Total unique items in the dataset: 100
Unique items recommended by the system: 97
Coverage: 0.9700
Note: A coverage less than 1.0 means the system cannot recommend all poss

```

✓ 2. Diversity

Diversity measures how dissimilar the recommended items are from each other. A higher diversity score indicates that the recommendations are not all very similar, which can be beneficial for user satisfaction. We'll measure this by averaging the pairwise similarity between recommended items for a given input.

```

# Function to calculate diversity for a set of recommendations
def calculate_diversity(recommended_isbns, item_similarity_df):
    if len(recommended_isbns) <= 1:
        return 0.0 # Diversity is 0 if there's only one or no recommendations

    # Filter the similarity matrix to include only the recommended items
    sub_matrix = item_similarity_df.loc[recommended_isbns, recommended_isbns]

    # Calculate the average pairwise dissimilarity (1 - similarity)
    # Exclude self-similarity (diagonal) by setting it to NaN before averaging
    np.fill_diagonal(sub_matrix.values, np.nan)
    # Average similarity among the recommended items
    average_similarity = sub_matrix.mean().mean()

    # Diversity is often defined as 1 - average_similarity
    diversity = 1 - average_similarity
    return diversity

# Get recommendations for a few example books and calculate their diversity
example_isbns_for_diversity = user_item_matrix.columns[[0, 1, 2]] # Use the fi

print("Calculating diversity for example recommendations:")
diversity_scores = {}

```

```

for isbn in example_isbns_for_diversity:
    recommendations_for_book = get_book_recommendations(isbn, item_similarity_df)
    if not recommendations_for_book.empty:
        rec_list = recommendations_for_book.index.tolist()
        div = calculate_diversity(rec_list, item_similarity_df)
        diversity_scores[isbn] = div
        print(f" - Book {isbn}: Diversity = {div:.4f}")
    else:
        print(f" - Book {isbn}: No recommendations to calculate diversity.")

# Overall average diversity for these examples
if diversity_scores:
    avg_diversity = sum(diversity_scores.values()) / len(diversity_scores)
    print(f"\nAverage diversity across example recommendations: {avg_diversity}")
else:
    print("No diversity scores calculated.")

```

[Show hidden output](#)

✓ 3. Novelty

Novelty measures how often the recommended items are exposed to all users. It tries to capture how 'surprising' or 'non-obvious' the recommendations are. A common way to approximate novelty is by using the inverse of an item's popularity. Less popular items, when recommended, are considered more 'novel'.

We'll define popularity as the number of ratings an item has received.

```

# Calculate item popularity (number of ratings per ISBN) from the processed Data
item_popularity = df_processed['ISBN'].value_counts()

# To avoid division by zero or very large inverse values for rare items, we can
# or use a log transformation or similar. For simplicity, we'll use raw counts
# A common approach for novelty is to sum the (1 - popularity_score) for recommended items

# A simple inverse popularity measure for novelty
def calculate_novelty(recommended_isbns, item_popularity_series):
    if not recommended_isbns:
        return 0.0

    novelty_scores = []
    for isbn in recommended_isbns:
        # Get popularity. If item not in popularity list, assume very low popularity
        popularity = item_popularity_series.get(isbn, 1) # Default to 1 to avoid division by zero
        # Inverse of popularity (e.g., 1/log(popularity+1) or similar)
        novelty_score = 1 / (np.log1p(popularity)) # log1p handles popularity=0
        novelty_scores.append(novelty_score)

    return np.mean(novelty_scores)

print("Calculating novelty for example recommendations:")
novelty_scores = {}
for isbn in example_isbns_for_diversity: # Re-using the same example books
    recommendations_for_book = get_book_recommendations(isbn, item_similarity_df)

```

```
if not recommendations_for_book.empty:
    rec_list = recommendations_for_book.index.tolist()
    nov = calculate_novelty(rec_list, item_popularity)
    novelty_scores[isbn] = nov
    print(f" - Book {isbn}: Novelty = {nov:.4f}")
else:
    print(f" - Book {isbn}: No recommendations to calculate novelty.")

# Overall average novelty for these examples
if novelty_scores:
    avg_novelty = sum(novelty_scores.values()) / len(novelty_scores)
    print(f"\nAverage novelty across example recommendations: {avg_novelty:.4f}")
else:
    print("No novelty scores calculated.")
```

Calculating novelty for example recommendations:

- Book 0060502258: Novelty = 0.2081
- Book 0060928336: Novelty = 0.2060
- Book 0060930535: Novelty = 0.2017

Average novelty across example recommendations: 0.2053

#6. Data Visualization Dashboard

#a. Load data

```
df=pd.read_csv('/content/country_wise_latest.csv')
df
```

	Country/Region	Confirmed	Deaths	Recovered	Active	New cases	New deaths
0	Afghanistan	36263	1269	25198	9796	106	10
1	Albania	4880	144	2745	1991	117	6
2	Algeria	27973	1163	18837	7973	616	8
3	Andorra	907	52	803	52	10	0
4	Angola	950	41	242	667	18	1
...	...	...	...	...	...	...	...
182	West Bank and Gaza	10621	78	3752	6791	152	2
183	Western Sahara	10	1	8	1	0	0
184	Yemen	1691	483	833	375	10	4
185	Zambia	4552	140	2815	1597	71	1
186	Zimbabwe	2704	36	542	2126	192	2

187 rows × 8 columns

#b. Data Cleaning and Preprocessing

```
print(df.info())
print(df.isnull().sum())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 187 entries, 0 to 186
Data columns (total 8 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Country/Region         187 non-null    object
1   Confirmed              187 non-null    int64
2   Deaths                187 non-null    int64
```

3	Recovered	187 non-null	int64
4	Active	187 non-null	int64
5	New cases	187 non-null	int64
6	New deaths	187 non-null	int64
7	New recovered	187 non-null	int64
8	Deaths / 100 Cases	187 non-null	float64
9	Recovered / 100 Cases	187 non-null	float64
10	Deaths / 100 Recovered	187 non-null	float64
11	Confirmed last week	187 non-null	int64
12	1 week change	187 non-null	int64
13	1 week % increase	187 non-null	float64
14	WHO Region	187 non-null	object

dtypes: float64(4), int64(9), object(2)

memory usage: 22.0+ KB

None

Country/Region 0

Confirmed 0

Deaths 0

Recovered 0

Active 0

New cases 0

New deaths 0

New recovered 0

Deaths / 100 Cases 0

Recovered / 100 Cases 0

Deaths / 100 Recovered 0

Confirmed last week 0

1 week change 0

1 week % increase 0

WHO Region 0

dtype: int64

#Preprocessing data

✓ Handling Duplicate Data

```
print(f"Number of duplicate rows before dropping: {df.duplicated().sum()}")
df.drop_duplicates(inplace=True)
print(f"Number of duplicate rows after dropping: {df.duplicated().sum()}")
```

Number of duplicate rows before dropping: 0

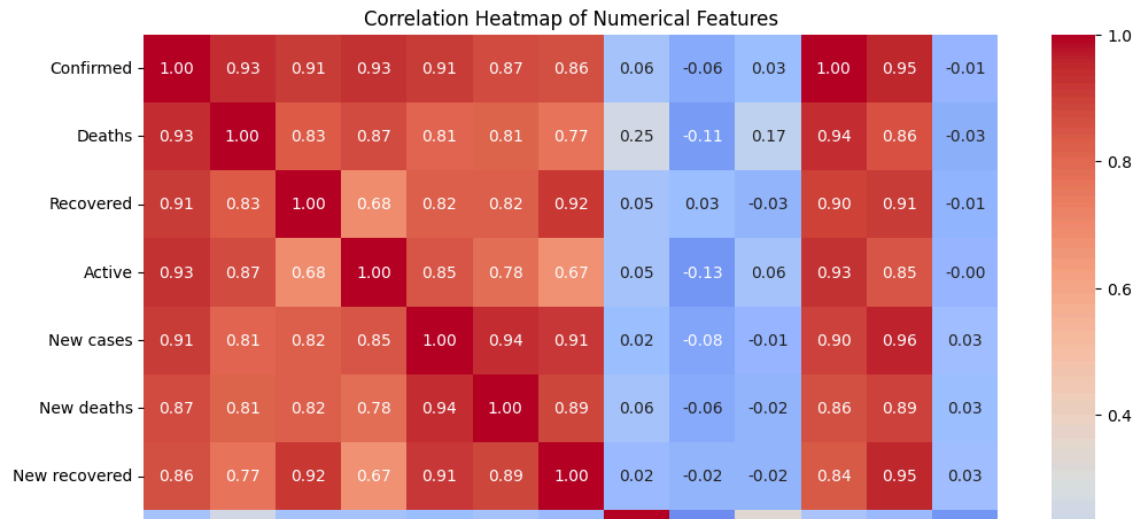
Number of duplicate rows after dropping: 0

#c. Visualization Design

1### Correlation Heatmap

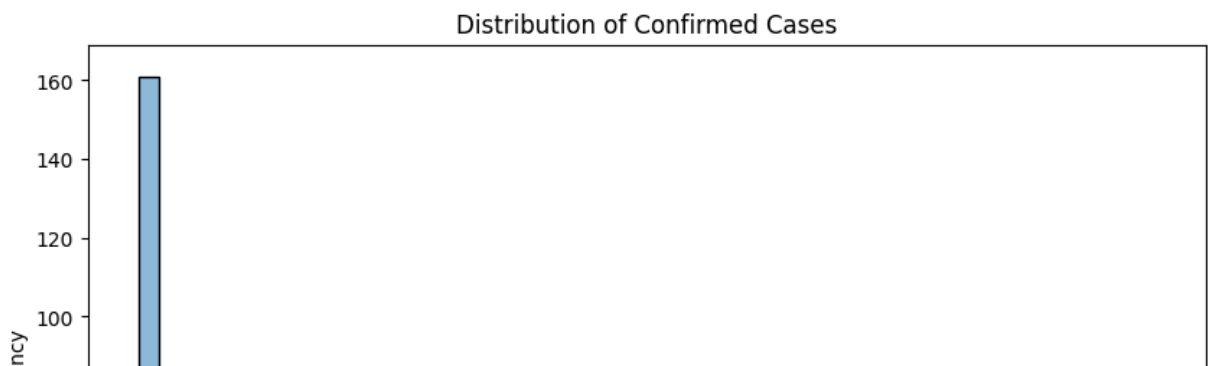
```
plt.figure(figsize=(12, 10))
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='coolwarm', fmt='.2f')
```

```
plt.title('Correlation Heatmap of Numerical Features')
plt.show()
```



✓ Distribution of Confirmed Cases

```
plt.figure(figsize=(10, 6))
sns.histplot(df['Confirmed'], kde=True, bins=50)
plt.title('Distribution of Confirmed Cases')
plt.xlabel('Confirmed Cases')
plt.ylabel('Frequency')
plt.show()
```



✓ Top 10 Countries by Confirmed Cases

```
top_10_confirmed = df.nlargest(10, 'Confirmed')

plt.figure(figsize=(12, 7))
sns.barplot(x='Country/Region', y='Confirmed', data=top_10_confirmed, palette=
plt.title('Top 10 Countries by Confirmed Cases')
plt.xlabel('Country/Region')
plt.ylabel('Confirmed Cases')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```

```
/tmp/ipykernel_12443/1765315041.py:4: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in a future version.

```
sns.barplot(x='Country/Region', y='Confirmed', data=top_10_confirmed, palette='dark')
```



✓ Top 10 Countries by Deaths

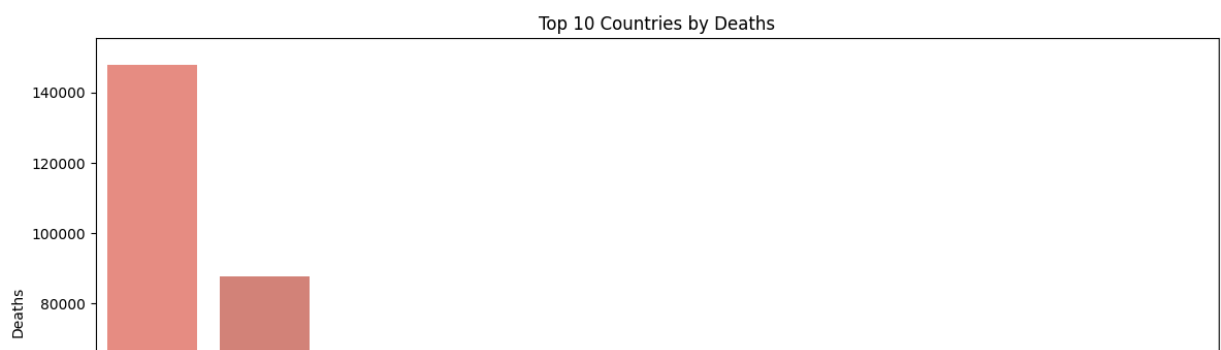
```
top_10_deaths = df.nlargest(10, 'Deaths')
```

```
plt.figure(figsize=(12, 7))
sns.barplot(x='Country/Region', y='Deaths', data=top_10_deaths, palette='dark')
plt.title('Top 10 Countries by Deaths')
plt.xlabel('Country/Region')
plt.ylabel('Deaths')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```

```
/tmp/ipykernel_12443/2379930040.py:4: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in a future version.

```
sns.barplot(x='Country/Region', y='Deaths', data=top_10_deaths, palette='dark')
```



✓ Confirmed Cases by WHO Region

```
who_region_cases = df.groupby('WHO Region')['Confirmed'].sum().sort_values(ascending=True)
plt.figure(figsize=(12, 7))
```

```
sns.barplot(x=who_region_cases.index, y=who_region_cases.values, palette='plasma')
plt.title('Confirmed Cases by WHO Region')
plt.xlabel('WHO Region')
plt.ylabel('Total Confirmed Cases')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```

/tmp/ipykernel_12443/673492561.py:4: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in a future version.

```
sns.barplot(x=who_region_cases.index, y=who_region_cases.values, palette='plasma')
```



✓ Deaths by WHO Region

#d. Dashboard Creation

✓ d. Dashboard Creation

I'll use `dash` to create an interactive dashboard, integrating the visualizations created earlier.

```
who_region_deaths = df.groupby('WHO Region')['Deaths'].sum().sort_values(ascending=True)

plt.figure(figsize=(12, 7))
sns.barplot(x=who_region_deaths.index, y=who_region_deaths.values, palette='magma')
plt.title('Deaths by WHO Region')
plt.xlabel('WHO Region')
plt.ylabel('Total Deaths')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```

